



UNIVERSITY OF GENEVA

Faculty of Science

Timing Attacks

Advanced Security

14X

Michel Jean Joseph Donnet

2026-05-12



Contents

1	Introduction	3
2	Timing attacks on RSA	3
2.1	RSA basics	3
2.2	Variance-based timing attack	4
2.2.1	Square-and-multiply algorithm	4
2.2.2	Exploiting timing variations	5
2.2.3	Mathematical analysis	6
2.2.3.1	Probability of a correct guess	6
2.2.3.2	Variance analysis	6
2.2.3.3	Impact of previous errors on the variance	7
2.2.4	Code implementation	8
2.2.4.1	Sources of noise	8
2.2.4.1.1	CPU cache effects	8
2.2.4.1.2	Branch prediction	8
2.2.4.1.3	Garbage collection	9
2.2.4.1.4	Multiplication optimizations	9
2.2.4.1.5	Other optimizations	9
2.2.4.2	Reducing the impact of noise	9
2.2.4.2.1	Reducing the impact of the garbage collector	10
2.2.4.2.2	Averaging timing measurements	11
2.2.4.2.3	Warm-up and optimizations of parameters	12
2.2.4.3	Performance of the attack	14
2.3	Pearson-based timing attack	15
2.3.1	Pearson correlation coefficient	16
2.3.2	Designing the model of expected timings	16
2.3.3	Advantages of the Pearson-based approach	17
2.3.4	Disadvantages of the Pearson-based approach	17
2.3.5	Results of the Pearson-based approach	18
2.3.6	Why the timing attack on RSA is still relevant	19
3	Fermat's factorization method on RSA	21
3.1	Security of RSA and factoring large integers	21
3.2	Description of Fermat's factorization method	21
3.3	From theory to the algorithm	22
	Bibliography	23



1 Introduction

The cryptography goal is to ensure the confidentiality, integrity, and authenticity of information, and it has become increasingly important in the digital age with the rise of the internet and digital communication where sensitive information is frequently transmitted. To achieve these goals, various cryptographic algorithms and protocols have been developed, including symmetric and asymmetric encryption, hashing, and digital signatures. These algorithms are based on mathematical problems that are computationally difficult to solve, such as factoring large integers or computing discrete logarithms, which provide the basis for their security since they are believed to be infeasible to break with current computational resources.

However, although these algorithms are designed to be secure against direct attacks, the implementation of these algorithms can introduce vulnerabilities that can be exploited by attackers. The vulnerabilities reside in the way the algorithms are implemented and executed, rather than in the algorithms themselves. Indeed, attackers can exploit side channels, which are unintended information leaks that occur during the execution of cryptographic algorithms such as timing information, power consumption, electromagnetic emissions, or even sound. By analyzing these side channels, attackers can gain insights into the internal workings of the cryptographic algorithm and potentially recover sensitive information such as secret keys.

In this report, we will focus on timing attacks, which are a type of side-channel attack that exploits the time it takes for a cryptographic algorithm to execute. Timing attacks can be used to recover secret keys or other sensitive information by measuring the time it takes for a cryptographic operation to complete and analyzing the variations in timing based on different inputs or conditions.

2 Timing attacks on RSA

RSA is a widely used asymmetric encryption algorithm that relies on the difficulty of factoring large integers to provide security. It allows in particular for sharing a symmetric key securely over an insecure channel, enabling secure communication between parties without the need for a pre-shared secret key.

The base principle of RSA resides in the use of a pair of keys: a public key for encryption and a private key for decryption. The RSA algorithm consists of three main steps: key generation, encryption, and decryption.

2.1 RSA basics

The key generation process involves selecting two large prime numbers, p and q , and computing their product $n = p \times q$, which serves as the modulus for both the public and private keys. The choice of p and q is crucial for the security of RSA, as the difficulty of factoring n into its prime factors is what provides the security of the algorithm. Indeed, if an attacker can factor n into p and q , they can compute the private key and break the encryption.



The algorithm of key generation can be summarized as follows:

1. Choose two distinct large prime numbers p and q .
2. Compute $n = p \times q$ and $\varphi(n) = (p - 1)(q - 1)$.
3. Choose an integer e such that
 - $1 < e < \varphi(n)$
 - $\gcd(e, \varphi(n)) = 1$
4. Compute d such that $e \cdot d \equiv 1 \pmod{\varphi(n)}$.

The public key consists of the pair (n, e) , while the private key consists of the pair (n, d) .

The encryption process consists of taking a plaintext message m and computing the ciphertext $c = m^e \pmod{n}$ using the public key, and the decryption process consists of taking the ciphertext c and computing the plaintext message $m = c^d \pmod{n}$ using the private key.

For convenience, we will use the following notations throughout the report:

- n : the modulus, which is the product of two large prime numbers p and q .
- e : the public exponent, which is an integer that is coprime to $\varphi(n)$.
- d : the private exponent, which is an integer such that $e \cdot d \equiv 1 \pmod{\varphi(n)}$.
- m : the plaintext message, which is an integer that is less than n .
- c : the ciphertext, which is an integer that is less than n .

2.2 Variance-based timing attack

Before the exchange of symmetric keys, the client and the server need to perform an RSA encryption and decryption operation to establish a secure communication channel. The timing of these operations can be exploited by attackers to infer information about the private key.

As mentioned earlier, the security of RSA relies on the difficulty of factoring large integers, but the implementation of RSA can introduce vulnerabilities that can be exploited by attackers. The goal of the attacker is to recover the unknown private key d thanks to the knowledge of the public key (n, e) and the ability to perform encryption and decryption operations on the server using the RSA algorithm.

2.2.1 Square-and-multiply algorithm

Timing attacks on RSA exploit the fact that the time it takes to perform certain operations in the RSA algorithm can vary based on the input values and the internal state of the algorithm. For example, the time it takes to perform the modular exponentiation operation $c^d \pmod{n}$ can vary based on the value of d and the input ciphertext c . Indeed, a simple implementation of the modular exponentiation operation can use a square-and-multiply algorithm, as described in the following python code:

```
def square_and_multiply(y, x, n):
    s = 1
    y %= n

    while x > 0:
        if (x % 2) == 1:
            s = (s * y) % n # Extra multiplication here when x is odd !
        x = x // 2
    return s
```



```

    y = (y * y) % n
    x = x >> 1

return s

```

This code computes $y^x \bmod n$ using the square-and-multiply algorithm, which is an efficient method for performing modular exponentiation. Indeed, computing $y^x \bmod n$ for very large values of x is computationally expensive.

The base idea of this algorithm is that computing y^x can be done by taking the binary representation of x , which is basically the decomposition of x into powers of 2, and then iteratively computing $y^x \bmod n$ by taking at each step the previous result and multiplying it by y if the current bit of x is 1, and then squaring the input y .

For instance, suppose we want to compute $6^{13} \bmod 17$.

- The binary representation of 13 is 1101, which corresponds to the powers of 2: $2^3 + 2^2 + 2^0$.
- We start with $s = 1$ and $y = 6$.
- For the first bit (1), we compute
 - $s = (s \cdot y) \bmod 17 = (1 \cdot 6) \bmod 17 = 6$
 - $y = y^2 \bmod 17 = 6^2 \bmod 17 = 2$.
- For the second bit (0), we compute
 - s remains unchanged since the bit is 0, so $s = 6$.
 - $y = y^2 \bmod 17 = 2^2 \bmod 17 = 4$. Note that here, y^2 corresponds to $y^{2^2} = y^{2 \cdot 2} = y^4$.
- For the third bit (1), we compute
 - $s = (s \cdot y) \bmod 17 = (6 \cdot 4) \bmod 17 = 24 \bmod 17 = 7$
 - $y = y^2 \bmod 17 = 4^2 \bmod 17 = 16$.
- For the fourth bit (1), we compute
 - $s = (s \cdot y) \bmod 17 = (7 \cdot 16) \bmod 17 = 112 \bmod 17 = 15$
 - $y = y^2 \bmod 17 = 16^2 \bmod 17 = 256 \bmod 17 = 1$.

So the result of $6^{13} \bmod 17$ is 15. The efficiency of the algorithm comes from the fact that it iteratively computes $y^x \bmod n$ by performing small multiplication and squaring operations thanks to the modular properties instead of performing a single large exponentiation operation.

But the timing of this algorithm can vary based on the value of the exponent x and the input y . Indeed, as seen in the above example, the algorithm performs an extra multiplication only when the bit of x is 1, which can lead to a longer execution time compared to when the bit is 0. This variation in timing is the basis for timing attacks on RSA.

2.2.2 Exploiting timing variations

Attackers can exploit the timing variations in the square-and-multiply algorithm to recover the private key d by measuring the time it takes for the server to perform decryption operations. Note that the attacker must know the modulus n , which is public.

First, the attacker sends multiple random ciphertexts c to the server and measures the time it takes for the server to perform the decryption operation $c^d \bmod n$.



Then for each bit of the private key d , the attacker guesses the value of the bit (0 or 1) and computes the expected timing of the decryption operation based on the guess for all the ciphertexts c .

Finally, the attacker subtracts his obtained timing measurements from the timing measurements obtained from the server and analyzes the variance of the obtained results. If the variance is low, it means that the guess for the bit is correct, while if the variance is high, it means that the guess for the bit is incorrect.

By repeating this process for all the bits of the private key d , the attacker can recover the entire private key.

2.2.3 Mathematical analysis

2.2.3.1 Probability of a correct guess

We consider N random ciphertexts $c_1, c_2, \dots, c_N$ that the attacker sends to the server for decryption.

Let T_i be the time taken for the server to perform the decryption operation $c^d \bmod n$ for a given ciphertext c_i .

Let x_b be the guess for the b first bits of the private key d .

Let $t(c_i, x_b)$ be the time taken to perform the decryption operation according to the guess x_b for the b first bits of d .

If the guess x_b is correct, the timing error $T_i - t(c_i, x_b)$ will be small, indicating a good match between the observed and expected timings. We define F as the probability to observe the timing error $T_i - t(c_i, x_b)$ if the guess x_b is correct. As each timing measurement is independent, the probability of a correct guess for the b first bits of d is proportional to the product of the probabilities of F for all the ciphertexts c_i :

$$P(x_b) \propto \prod_{i=1}^N F(T_i - t(c_i, x_b))$$

Suppose that the $b - 1$ bits of d are correct, and we want to guess the value of the b -th bit. We note x_{b_0} and x_{b_1} the guesses for the b -th bit being 0 and 1, respectively.

The probability of having x_{b_1} is:

$$P(x_{b_1}) = \frac{\prod_{i=1}^N F(T_i - t(c_i, x_{b_1}))}{\prod_{i=1}^N F(T_i - t(c_i, x_{b_0})) + \prod_{i=1}^N F(T_i - t(c_i, x_{b_1}))}$$

If $P(x_{b_1}) > 0.5$, it means that the guess x_{b_1} is more likely to be correct than the guess x_{b_0} , and thus we can conclude that the b -th bit of d is likely to be 1.

Concretely, computing the exact distribution of F is not feasible.

2.2.3.2 Variance analysis

Instead of computing the exact distribution of F , we can analyze the variance of the timing errors for the two guesses x_{b_0} and x_{b_1} .



Suppose the private key d has l bits. The time taken for the decryption operation can be denoted as $T_i = \sum_{k=1}^l t_k + \varepsilon$ with t_k the time taken for the k -th step of the algorithm and ε representing the timing noise caused by various factors such as system load, network latency, or other sources of randomness.

Suppose that x_{b-1} is known to be correct. The attacker can compute the time taken for each step of the first $b-1$ bits as $\sum_{k=1}^{b-1} t_k$.

The time difference is then:

$$T_i - t(c_i, x_{b-1}) = \sum_k^l t_k + \varepsilon - \sum_k^{b-1} t_k = \sum_{k=b}^l t_k + \varepsilon$$

If the guess x_b is correct, the time difference will be:

$$T_i - t(c_i, x_b) = \sum_{k=b+1}^l t_k + \varepsilon$$

And if the guess x_b is incorrect, the time difference will be:

$$T_i - t(c_i, x_b) = \sum_k^l t_k + \varepsilon - \left(\sum_k^{b-1} t_k + t_{b_{\text{incorrect}}} \right) = \sum_{k=b+1}^l t_k + \varepsilon + (t_{b_{\text{correct}}} - t_{b_{\text{incorrect}}})$$

.

We can constate that the time difference for the correct guess x_b is smaller than the time difference for the incorrect guess x_b by a factor of $t_{b_{\text{correct}}} - t_{b_{\text{incorrect}}}$, meaning that the variance of the time differences for the correct guess x_b will be smaller than the variance of the time differences for the incorrect guess x_b .

So by analyzing the variance of the time differences for the two guesses x_{b_0} and x_{b_1} , the attacker can determine which guess is more likely to be correct, and thus recover the bits of the private key d one by one.

2.2.3.3 Impact of previous errors on the variance

Now, suppose that the c -th bit of d is incorrect for some $c < b-1$. The time difference for the correct guess x_b will be:

$$T_i - t(c_i, x_b) = \sum_k^l t_k + \varepsilon - \left(\sum_k^{c-1} t_k + \sum_{k=c}^b t_k \right) = \sum_{k=b+1}^l t_k + \varepsilon + \left(\sum_{k=c}^b (t_{k_{\text{correct}}} - t_{k_{\text{incorrect}}}) \right)$$

We have t_k which starts to be different for the correct and incorrect guesses for all the bits from c to b since the intermediate steps of the algorithm will be executed differently due to the error in the guess for the c -th bit, leading to a different t_k for all the bits from c to b even if the guess for the b -th bit is correct.

So when the guess x_b has some previous errors, the variance of the correct guess will start to be similar to the variance of the incorrect guess.

This property can be exploited to detect errors in the guess for the b -th bit. Indeed, attacker can keep track of the variance of the time differences and come back to a previous guess if



the variance starts to be too similar for both the correct and incorrect guesses, indicating that there might be an error in the previous bits of the guess.

2.2.4 Code implementation

The code implementation was done first in Python, and then in Rust and C++ to try to reduce the noise in the timing measurements and thus increase the chances of success of the attack.

Indeed, the python implementation was not successful at all in recovering the private key due to the high noise caused by the Python interpreter and its optimizations, which totally masked the timing variations caused by the square-and-multiply algorithm.

2.2.4.1 Sources of noise

There are several sources of noise that can affect the timing measurements in programming languages, especially in high-level languages like Python.

2.2.4.1.1 CPU cache effects

CPU cache effects that can cause variability in the timing measurements based on the memory access patterns of the algorithm.

First, there are different levels of CPU cache (L1, L2, L3) that can store recently accessed data and instructions.

These caches are designed to speed up access to frequently used data and instructions, but they can also introduce variability in the timing measurements. Indeed, the cache L1 is the fastest but also the smallest, while the cache L3 is the slowest but also the largest, meaning that if the data or instructions needed for the algorithm are in the cache L1, the timing measurements will be faster compared to when they are in the cache L3 or not in the cache at all.

If the data or instructions needed are not in the cache, the CPU triggers a cache miss, which means that it has to fetch the data from the main memory, leading to a significant delay in the execution of the algorithm and thus in the timing measurements.

These cache effects represent a significant source of noise in timing measurements.

2.2.4.1.2 Branch prediction

Branch prediction is a technique used by modern CPUs to improve performance by guessing the outcome of conditional statements and executing instructions based on those guesses. When the CPU encounters a conditional statement (e.g., an if statement), it makes a guess about which branch of the code will be executed next based on past behavior and patterns. This allows the CPU to continue executing instructions without waiting for the outcome of the conditional statement, which can improve performance.

So if the CPU correctly predicts the branch, it can continue executing instructions without interruption, but if the CPU incorrectly predicts the branch, it has to discard the incorrectly executed instructions and fetch the correct instructions, leading to a significant delay in the execution of the algorithm and thus in the timing measurements.



That's why branch prediction can cause variability in the timing measurements based on the input values and the internal state of the algorithm, as different inputs can lead to different execution paths and thus different branch predictions.

2.2.4.1.3 Garbage collection

Garbage collection is a form of automatic memory management that is used in many programming languages, including Python. It is responsible for automatically freeing up memory that is no longer in use by the program, which can help prevent memory leaks and improve performance.

However, the garbage collector can be triggered at any time during the execution of the program. When the garbage collector runs, it can cause significant delays in the execution of the program: it has to pause the execution of the program, scan the memory for objects that are no longer in use, and free up the memory occupied by those objects.

This can lead to significant variability in the timing measurements since the garbage collector can be triggered at different times during the execution of the algorithm, leading to different timing measurements for the same operations.

2.2.4.1.4 Multiplication optimizations

The usage of modern programming languages introduces various optimizations that can affect the timing measurements, such as the optimization of multiplication operations.

The multiplication of large integers can be optimized using different algorithms depending on the size of the numbers. For small integers, the standard multiplication algorithm is used whereas for larger integers, more efficient algorithms such as Karatsuba or Toom-Cook can be used.

These optimizations can lead to different timing measurements for the same operations based on the size of the numbers being multiplied.

2.2.4.1.5 Other optimizations

There are also other optimizations that can be introduced by the programming language or the compiler, such as loop unrolling, instruction reordering, or just-in-time compilation, which can further introduce variability in the timing measurements.

So the sources of noise in timing measurements can be quite significant, especially in high-level programming languages like Python, and they can totally mask the timing variations caused by the square-and-multiply algorithm.

2.2.4.2 Reducing the impact of noise

There are several techniques that can be used to try to reduce the impact of these sources of noise in timing measurements.

Inspired by the blog post “Kocher’s Timing Attack: A Journey from Theory to Practice” [1], I try to implement some of them, trying to mitigate the impact of noise and thus increase the chances of success of the attack.



2.2.4.2.1 Reducing the impact of the garbage collector

As mentioned in Section 2.2.4.1, the garbage collector is a significant source of noise in the timing measurements, so one way to reduce the noise is to disable the garbage collector during the timing measurements.

This can be done manually using the `gc` module in Python, which provides functions to enable and disable the garbage collector. Even if this can help to reduce the noise caused by the garbage collector, it is not sufficient to extract the signal from the noise and thus distinguish between the case with the extra multiplication (key 1) and the case without the extra multiplication (key 0) since there are still other sources of noise that can affect the timing measurements.

However the right approach to perform the precise timing measurements was not to manually disable the garbage collector, but rather to use the `timeit` module in Python, which is designed to perform accurate timing measurements.

Indeed, the `timeit` module, which is a part of the Python standard library, is specifically designed for measuring the execution time of small code snippets with high precision. It automatically handles various sources of noise, including disabling the garbage collector during the timing measurements, which can help to reduce the impact of the garbage collector on the timing measurements and thus increase the chances of success of the attack.

The Figure 1 shows the comparison of timing measurements with manual management of the garbage collector (GC) and using the `timeit` module, which handles GC automatically. It is done by repeating the timing measurements for a fixed ciphertext and fixed key, and then plotting the distribution of the timing measurements for both approaches. We can easily see that the timing measurements obtained with the `timeit` module are much more consistent and less noisy compared to the timing measurements obtained with manual management of the garbage collector.

Indeed, with the `timeit` module, the timing measurements are more concentrated around a certain value since the pick of frequency of the timing measurements with same value is around 10 times higher than with the manual management of the garbage collector, which means that the timing measurements obtained with the `timeit` module are more consistent and less affected by noise compared to the timing measurements obtained with manual management of the garbage collector.

On top of that, the minimum timing measurement obtained with the `timeit` module is 100 times lower than the minimum timing measurement obtained with manual management of the garbage collector, which means that the `timeit` module is able to capture the true decryption time more accurately compared to the manual management of the garbage collector. This means that the `timeit` module allows to reduce more noise than only the noise caused by the garbage collector, and thus is a much better approach to perform the timing measurements for the attack compared to the manual management of the source of noise. That's why I used the `timeit` module for the timing measurements in the attack implementation instead of manually disabling the garbage collector.

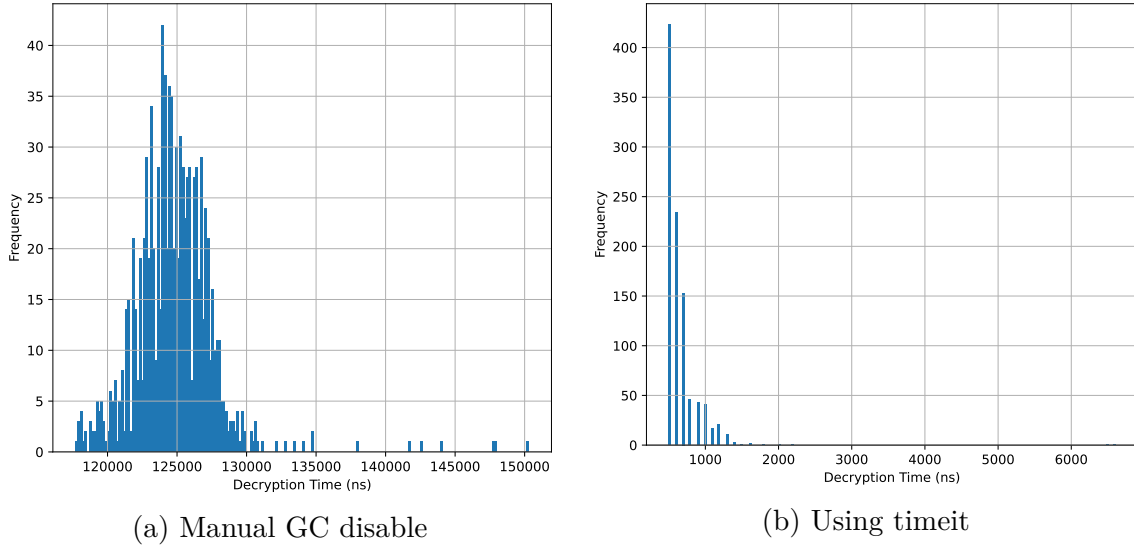


Figure 1: Comparison of timing measurements with manual garbage collector (GC) disable versus using timeit (GC handled automatically)

2.2.4.2.2 Averaging timing measurements

Once the garbage collector is disabled, there are still other sources of noise that can affect the timing measurements, as described in Section 2.2.4.1.

The most common approach to try to extract the signal from the noise in timing measurements is to perform a large number of timing measurements and then average them. Indeed, by performing a large number of timing measurements and then averaging them, we can reduce the impact of random fluctuations in the timing measurements.

However, as shown on Figure 1, there are some samples for which the timing measurements are significantly higher than the others due to the sources of noise described in Section 2.2.4.1, which can skew the average and thus make it not representative of the true decryption time.

To solve this problem, the first idea was to remove the outliers from the timing measurements by filtering out the timing measurements that are too far from the center of the distribution, such as below 25% and above 75%, which can help to reduce the impact of outliers on the analysis of the timing measurements by keeping only the most representative samples.

But is the averaging of the timing measurements sufficient to extract the signal from the noise and thus distinguish between the case with the extra multiplication (key 1) and the case without the extra multiplication (key 0) ? Of course, not. It was my first approach to try reducing the impact of noise, but it was not sufficient to extract the signal from the noise and thus distinguish between the two cases. Indeed, when I was using a manual approach to disable the garbage collector, I was not able to distinguish between the two cases even after removing the outliers from the timing measurements, which can be explained by the fact that there are still other sources of noise that can affect the timing measurements, making the averaging of the timing measurements not sufficient to extract the signal from the noise.

In fact, the noise in the timing measurements is an additive noise, meaning that it can only add latency to the timing measurements, but it can never subtract latency from the timing measurements. This means that the minimum timing measurement is the best estimator



of the true decryption time, as the minimum timing measurement is the one that is least affected by the noise, while the other timing measurements can be significantly affected by the noise and thus can be much higher than the true decryption time. So contrary to the average that can be skewed by outliers, the minimum timing measurement is the right approach to extract the signal from the noise.

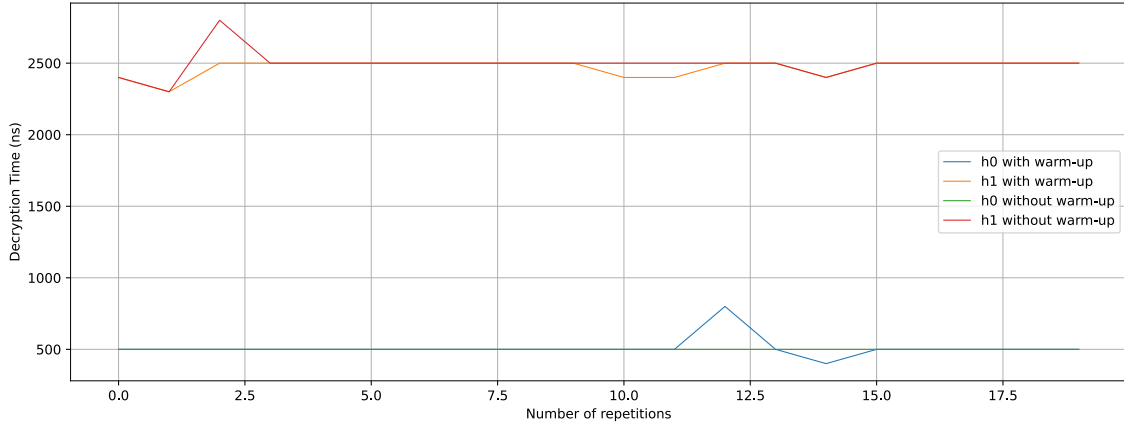
2.2.4.2.3 Warm-up and optimizations of parameters

To reduce the impact of noise caused by the CPU cache effects and branch prediction, a warm-up phase was added before the timing measurements.

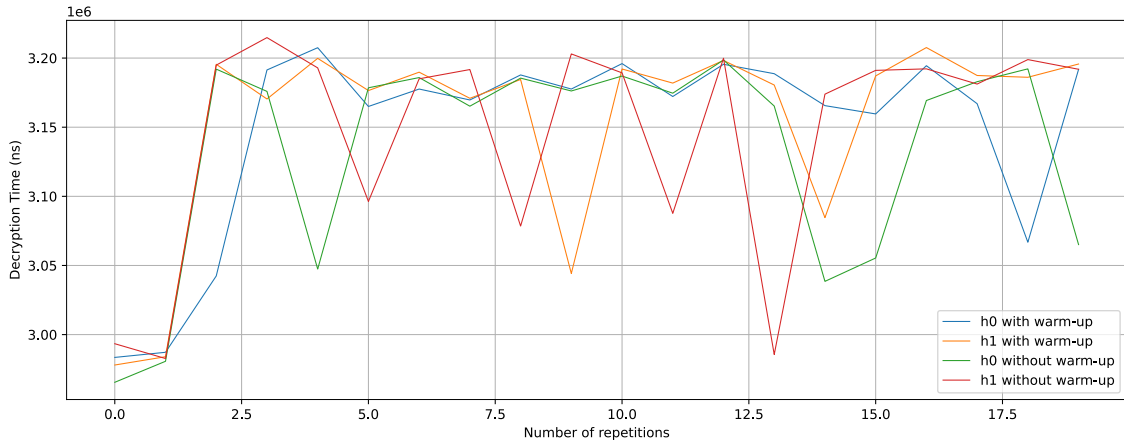
The warm-up phase consists in performing a certain number of decryption operations before the timing measurements. This allows the CPU to load the relevant data and instructions into the cache and the branch predictor to learn the execution patterns of the algorithm, which can help to reduce the impact of noise caused by these factors and thus increase the chances of success of the attack.

However, by looking at the results with and without the warm-up phase in Figure 2, we can see that the warm-up phase does not have a significant impact on the timing measurements, and thus does not help to reduce the impact of noise caused by the CPU cache effects and branch prediction. Perhaps that the `timeit` module already handles some of the optimizations that can be achieved with a warm-up phase, such as loading the relevant data and instructions into the cache and learning the execution patterns of the algorithm, which can explain why the warm-up phase does not have a significant impact on the timing measurements.

So the attack will be implemented without a warm-up phase, as it does not seem to have a significant impact on the timing measurements and thus does not help to reduce the impact of noise caused by the CPU cache effects and branch prediction. This allows to reduce the time taken for the attack, as the warm-up phase can be quite time-consuming, especially if we want to perform a large number of repetitions of the timing measurements.



(a) Small key size



(b) Big key size

Figure 2: Comparison of differences with (h1) and without (h0) extra multiplications with and without warm-up phase with minimum timing measurements for small and big key sizes

If we look at the results obtained with the big key size, we can see that it starts to be more difficult to distinguish between the two cases with and without the extra multiplication, which can be explained by the fact that the timing variations caused by the square-and-multiply algorithm are smaller for larger key sizes, making it more difficult to extract the signal from the noise and thus distinguish between the two cases. This can be problematic for the attack, as it relies on the ability to distinguish between the two cases with and without the extra multiplication to recover the bits of the private key d .

So I tried in Figure 3 to analyze the minimum repetitions needed to start to be able to distinguish between the two cases with and without the extra multiplication for a big key size, by plotting the minimum decryption time obtained for both cases with different numbers of repetitions of timing measurements. Each measurement is repeated 10 times to try to reduce the impact of noise, allowing to better visualize if the two cases are distinguishable or not.

The results show that for a big key size, it starts to be possible to distinguish between the two cases when we perform around 10000 repetitions of timing measurements, which can be quite time-consuming...

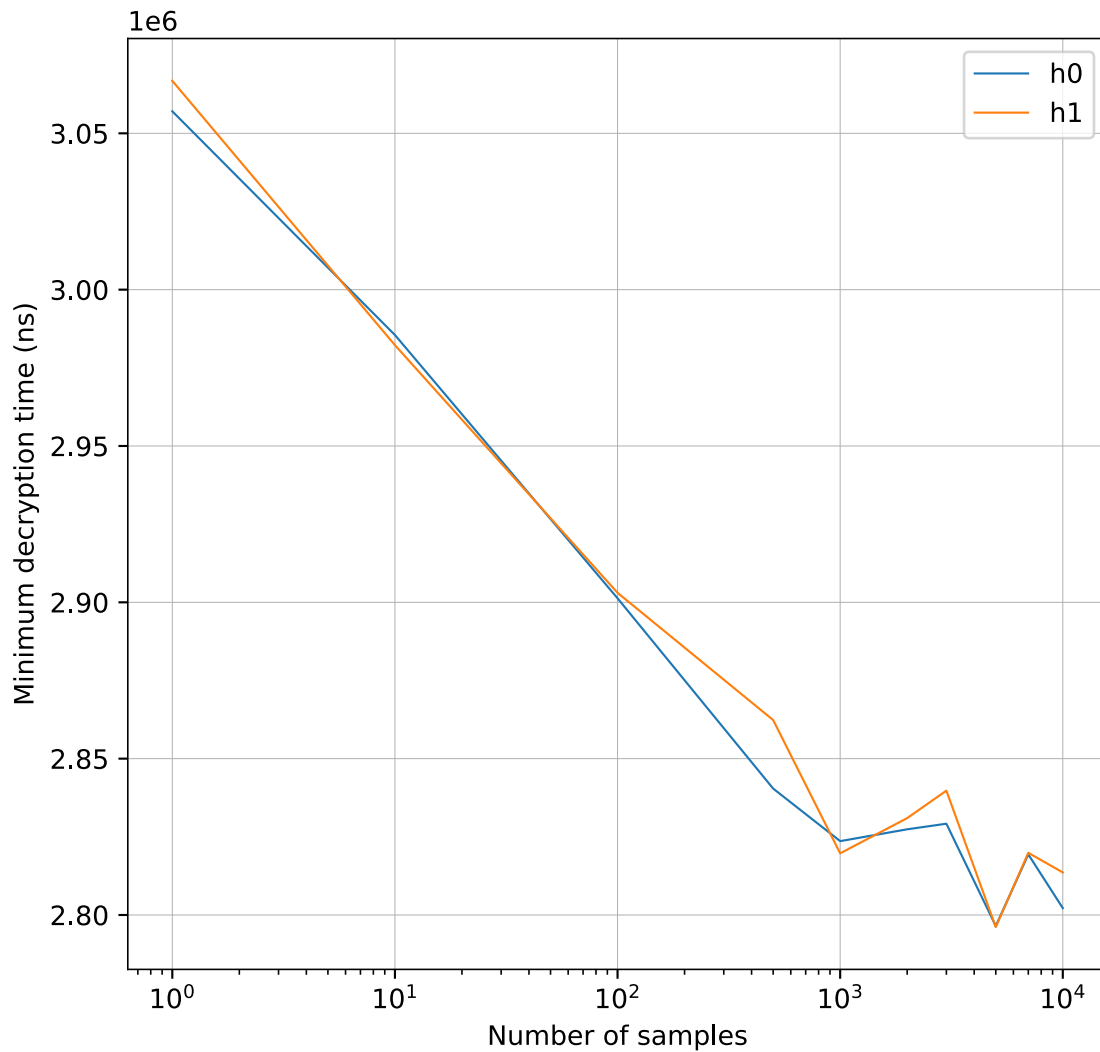


Figure 3: Convergence of minimum decryption time with number of samples

2.2.4.3 Performance of the attack

I first tried to implement the attack in Python with a simple version that does not look after noise, but it was not successful at all in recovering the private key due to the high noise caused by the Python interpreter and its optimizations, which totally masked the timing variations caused by the square-and-multiply algorithm. Indeed, the error rate was at best around 50%, meaning that the attack was not able to distinguish between the two cases and make only random guesses for the bits of the private key d .

Then, I implemented a version that uses repeated timing measurements and that looks after the garbage collector, but it was still not successful at all in recovering the private key. This is perhaps due to the fact that I was using the average of the timing measurements instead of the minimum. Note that I thought to remove the outliers from the timing measurements to try to reduce the impact of noise, but it was not sufficient to extract the signal from the noise.

Finally, I tried to implement the attack in Python with the improvements described above, and I run it for 13 hours to try to recover the private key d with a key size of 1024 bits. I only



managed to recover 5 bits of the private key d using 1000 messages and 10000 repetitions of timing measurements for each message. And I have assumed that the first 16 bits of the private key d are known. I use 3 candidates per iteration to try to reduce the impact of errors in the previous bits on the variance and thus reduce the number of errors in the recovered private key.

On the obtained results, we can see that one of the candidates has an error rate of 0%, meaning that it is the correct private key, while the other candidates have an error rate of 20%, meaning that they have some errors in the recovered bits of the private key. This means that the attack was successful in recovering some bits of the private key. But due to the computational cost of the attack, I was not able to recover more than 5 bits of the private key d in a reasonable time.

```
Iteration 1/5: bit guessed = 1 (var h0 = 1.5563e+08, var h1 = 1.5628e+08) ✗
WRONG
```

```
Iteration 2/5: bit guessed = 1 (var h0 = 1.5586e+08, var h1 = 1.5514e+08) ✓
CORRECT
```

```
Iteration 3/5: bit guessed = 1 (var h0 = 1.5520e+08, var h1 = 1.5653e+08) ✓
CORRECT
```

```
Iteration 4/5: bit guessed = 0 (var h0 = 1.5497e+08, var h1 = 1.5614e+08) ✓
CORRECT
```

```
Iteration 5/5: bit guessed = 1 (var h0 = 1.5522e+08, var h1 = 1.5507e+08) ✓
CORRECT
```

Candidate #1

```
Key:      1234141
Variance: 1.5475e+08
Error rate: 20.00% (1/5 bits wrong)
```

Candidate #2

```
Key:      1496285
Variance: 1.5507e+08
Error rate: 0.00% (0/5 bits wrong)
```

Candidate #3

```
Key:      447709
Variance: 1.5522e+08
Error rate: 20.00% (1/5 bits wrong)
```

2.3 Pearson-based timing attack

To exploit more efficiently the timing variations in the square-and-multiply algorithm, it is possible to use the Pearson correlation coefficient instead of the variance to determine which guess for the bit of the private key d is more likely to be correct.



2.3.1 Pearson correlation coefficient

The Pearson correlation coefficient is a measure of the linear correlation between two variables, in this case, the timing measurements and the expected timings based on the guess for the bit of the private key d .

The Pearson correlation coefficient can be formally defined as:

$$r = \frac{\text{Cov}(T, t(C, x_b))}{\sigma_T \sigma_{t(C, x_b)}}$$

with:

- r the Pearson correlation coefficient.
- C the set of ciphertexts $C = \{c_1, c_2, \dots, c_n\}$.
- T the vector of timing measurements obtained from the server for the set of ciphertexts C .
- $t(C, x_b)$ the vector of expected timings based on the guess x_b for the bit of the private key d for the set of ciphertexts C .
- σ_T the standard deviation of the timing measurements T .
- $\sigma_{t(C, x_b)}$ the standard deviation of the expected timings $t(C, x_b)$.

A value close to 1 indicates a strong positive correlation, meaning that the timing measurements and the expected timings are closely related. A value close to -1 indicates a strong negative correlation, and a value close to 0 indicates no linear correlation.

As we want that our guess for the bit of the private key d explains the timing variations in the square-and-multiply algorithm, we want to maximize the Pearson correlation between the timing measurements and the expected timings.

Depending on the model of the expected timings based on the guess for the bit of the private key d , it is possible to have a positive or negative correlation between the timing measurements and the expected timings, meaning that the model of the expected timings is inverted compared to the timing measurements.

So we are more interested in the strength of the correlation rather than its sign, meaning that we want to maximize the absolute value of the Pearson correlation coefficient between the timing measurements and the expected timings.

2.3.2 Designing the model of expected timings

The idea is to create a function $t(C, x_b)$ that estimates the expected timings based on the guess x_b and the set of ciphertexts C . Indeed, the model must take into account the size of the ciphertexts that lead to different timings for the decryption operation, as well as the guess for the bit of the private key d that leads to different execution paths in the square-and-multiply algorithm and thus different timings.

A simple approach can consist to use the Hamming weight of the intermediate values in the square-and-multiply algorithm as a model for the expected timings. The Hamming weight of a binary number is the number of bits that are set to 1 in the binary representation of the number. It can be used to estimate the complexity of the operations, as the operations that involve more bits set to 1 can be more complex and thus take more time to execute.

The cost computation for the square-and-multiply algorithm can be modeled as follows:



```
def square_and_multiply_with_cost(y, x, n, t_mul=1.1):
    """
    Square-and-multiply algorithm with cost modeling.
    - t_mul: cost factor for multiplication operations
    - t_square: cost factor for squaring operations
    """
    s = 1
    y %= n
    cost = 0

    while x > 0:
        if x & 1:
            s = (s * y) % n
            cost += t_mul * s.bit_count()
        y = (y * y) % n
        x >>= 1
    return s, cost
```

Note that we increase the cost only for the extra multiplication operations, as the squaring operations are performed in both cases with and without the extra multiplication, meaning that they do not contribute to the timing variations that we want to exploit for the attack.

2.3.3 Advantages of the Pearson-based approach

Compared to the variance based approach, the Pearson correlation coefficient can be more efficient to determine which guess for the bit of the private key d is more likely to be correct, as it takes into account the relationship between the timing measurements and the expected timings based on the guess for the bit of the private key d , rather than just looking at the variance of the timing measurements for each guess.

Indeed, Pearson based approach aims to explain the variations in the execution time of the square-and-multiply algorithm using a simple model that is independent of noise, machine architecture, algorithm implementation, the chosen programming language, and so on.

In this way, the attack depends only on the ability to capture the timing variations from a server.

On top of that, the Pearson approach allows to avoid performing a large number of timing measurements for each guess, allowing to significantly reduce the time taken for the attack.

For instance, instead of performing $1000 \text{ messages} \cdot 10000 \text{ repetitions} = 10 \text{ million}$ timing measurements for each guess for the bit of the private key d , we only need to compute the Pearson correlation coefficient between two vectors of size 1000, which can be done in a reasonable time even for a large key size.

2.3.4 Disadvantages of the Pearson-based approach

The main disadvantage of the Pearson-based approach is that it relies on the design of a good model for the expected timings based on the guess for the bit of the private key d .

If the model is not well designed, it can lead to a low Pearson correlation coefficient even for the correct guess for the bit of the private key d , making it difficult to distinguish between



the correct and incorrect guesses. And if the model is too complex, the computation of the expected timings can be time-consuming, which can increase the time taken for the attack.

2.3.5 Results of the Pearson-based approach

The results of the Pearson-based approach are quite good, as the second candidate has no error like for the variance-based approach.

```
Iteration 1/5: bit guessed = 0 (r h0 = -0.0077, r h1 = -0.0238) ✓ CORRECT
Iteration 2/5: bit guessed = 0 (r h0 = -0.0233, r h1 = -0.0185) ✗ WRONG
Iteration 3/5: bit guessed = 1 (r h0 = -0.0216, r h1 = -0.0178) ✓ CORRECT
Iteration 4/5: bit guessed = 0 (r h0 = -0.0055, r h1 = -0.0056) ✓ CORRECT
Iteration 5/5: bit guessed = 1 (r h0 = -0.0024, r h1 = +0.0144) ✓ CORRECT
```

Candidate #1

```
Key:      2020573
Pearson r: +0.0144
Error rate: 20.00% (1/5 bits wrong)
```

Candidate #2

```
Key:      1496285
Pearson r: +0.0087
Error rate: 0.00% (0/5 bits wrong)
```

Candidate #3

```
Key:      316637
Pearson r: -0.0003
Error rate: 40.00% (2/5 bits wrong)
```

However, compared to the variance-based approach, only 2000 repetitions of timing measurements were used, and the attack was achieved in around 2 hours, which is significantly faster than the 13 hours taken for the variance-based approach.

We can constate that the Pearson correlation coefficient (Pearson r) is very close to 0 for all candidates, even for the correct candidate, which can be explained by the fact that the model for the expected timings is not perfect and thus does not explain well the timing variations in the square-and-multiply algorithm, or perhaps that the timing variations in the square-and-multiply algorithm don't vary so much.

To improve the Pearson-based approach, it would be interesting to try to design a better model for the expected timings based on the guess for the bit of the private key d , which can help to increase the Pearson correlation coefficient for the correct guess and thus make it easier to distinguish between the correct and incorrect guesses. Or it would be interesting to try to collect more timing measurements from the server to have more chance to capture the true timing variations.

A test to recover 10 bits was done, and the results are not so good because the error rate of the three first candidates is around 40%, meaning that they have some errors in the recovered bits of the private key, while the two last candidates have an error rate of 30%, meaning



that they have some errors in the recovered bits of the private key but less than the three first candidates.

An error rate of 40% is almost random guessing, meaning that the attack is not able to distinguish between the correct and incorrect guesses for the bits of the private key d , while an error rate of 30% is better than random guessing.

```
Iteration 1/10: bit guessed = 1 (r h0 = +0.0000, r h1 = +0.0461) ✓ CORRECT
Iteration 2/10: bit guessed = 0 (r h0 = +0.0461, r h1 = +0.0429) ✓ CORRECT
Iteration 3/10: bit guessed = 1 (r h0 = +0.0429, r h1 = +0.0704) ✓ CORRECT
Iteration 4/10: bit guessed = 0 (r h0 = +0.0704, r h1 = +0.0512) ✗ WRONG
Iteration 5/10: bit guessed = 1 (r h0 = +0.0512, r h1 = +0.0461) ✓ CORRECT
Iteration 6/10: bit guessed = 1 (r h0 = +0.0626, r h1 = +0.0657) ✗ WRONG
Iteration 7/10: bit guessed = 0 (r h0 = +0.0657, r h1 = +0.0756) ✗ WRONG
Iteration 8/10: bit guessed = 1 (r h0 = +0.0756, r h1 = +0.0687) ✓ CORRECT
Iteration 9/10: bit guessed = 1 (r h0 = +0.0930, r h1 = +0.0880) ✗ WRONG
Iteration 10/10: bit guessed = 0 (r h0 = +0.0880, r h1 = +0.0782) ✓ CORRECT
```

Candidate #1

Key: 407
 Pearson r: +0.1067
 Error rate: 40.00% (4/10 bits wrong)

Candidate #2

Key: 663
 Pearson r: +0.0962
 Error rate: 40.00% (4/10 bits wrong)

Candidate #3

Key: 919
 Pearson r: +0.0956
 Error rate: 50.00% (5/10 bits wrong)

Candidate #4

Key: 247
 Pearson r: +0.0930
 Error rate: 30.00% (3/10 bits wrong)

Candidate #5

Key: 151
 Pearson r: +0.0904
 Error rate: 30.00% (3/10 bits wrong)

2.3.6 Why the timing attack on RSA is still relevant

Even if the timing attack on RSA is not as efficient as I expected, it is still relevant to study it and try to implement it, as it allows to understand the vulnerabilities of RSA and the importance of implementing countermeasures against timing attacks. Indeed, the timing attack on RSA is a well-known attack that can be used to recover the private key d by exploiting the timing variations in the square-and-multiply algorithm, and it has been shown



to be effective in certain scenarios, such as when the implementation of RSA is not properly protected against timing attacks.

In general, servers are set up one time for many years, and they are optimized as much as the actual modern CPU architecture allows. On top of that, the implementations on a server are often done in low-level programming languages such as C or Rust, which can allow to reduce the noise in the timing measurements and thus increase the chances of success of the attack compared to high-level programming languages such as Python.

In this way, the timing variations caused by the square-and-multiply algorithm running on a server can be more significant and thus easier to exploit for the attack compared to the timing variations caused by the square-and-multiply algorithm running on a local machine with a high optimized setup.



3 Fermat's factorization method on RSA

3.1 Security of RSA and factoring large integers

The private key d in RSA is computed based on the prime factors p and q of the modulus n . So another way to recover the private key d is to factor the modulus n into its prime factors p and q , and then compute d using the formula $d \equiv e^{-1} \bmod \varphi(n)$ with $\varphi(n) = (p-1)(q-1)$ the Euler's totient function. The inverse of e modulo $\varphi(n)$ can be computed using the Extended Euclidean Algorithm, which is an efficient method for computing the greatest common divisor of two integers and their multiplicative inverse.

But factoring large integers is a computationally hard problem, and the security of RSA relies on this fact.

However, if p and q are close to each other, meaning that the difference between p and q is small, then it becomes easy to factor n using Fermat's factorization method, as described by Hanno Böck in his paper "Fermat Factorization in the Wild" [2].

This paper is quite recent (2023) and it shows that there are still some implementations of RSA that are vulnerable to Fermat's factorization method due to the fact that they use prime numbers p and q that are close to each other, which can be explained by the fact that it is easier to generate prime numbers that are close to each other compared to generating prime numbers that are far from each other.

That's why it is important to study Fermat's factorization method and try to implement it, as it allows to understand the vulnerabilities of RSA and the importance of generating prime numbers that are not close to each other to ensure the security of RSA.

3.2 Description of Fermat's factorization method

Fermat's factorization method relies on the fact that n is a product of two odd primes p and q . As p and q are odd, there always exist an integer number which is at the middle of p and q , which is $x = \frac{p+q}{2}$. Then, we consider y as the distance between x and p (or q):

$$y = x - p = q - x$$

So we have:

- $p = x - y$
- $q = x + y$

If we express n in terms of x and y , we get:

$$n = p \times q = (x - y)(x + y) = x^2 - y^2$$

From this equation, we can deduce that $y^2 = x^2 - n$.



3.3 From theory to the algorithm

The Fermat's factorization method consists in finding the smallest integer x such that $y^2 = x^2 - n$ is a perfect square, meaning that y is an integer.

Once we find such an integer x , we can compute y as $y = \sqrt{x^2 - n}$, and then we can obtain the prime factors p and q as described in Section 3.2.

So the algorithm can be summarized as follows:

1. Compute $x = \lceil \sqrt{n} \rceil$.
2. Compute $y^2 = x^2 - n$.
3. If y^2 is a perfect square, then $y = \sqrt{y^2}$ and we can compute $p = x - y$ and $q = x + y$.
4. Otherwise, increment x and repeat from step 2.



Bibliography

- [1] M. Ochoa, “Kocher's Timing Attack: A Journey from Theory to Practice.” Accessed: May 10, 2026. [Online]. Available: <https://blog.zksecurity.xyz/posts/timing-kocher/>
- [2] H. Böck, “Fermat Factorization in the Wild.” [Online]. Available: <https://eprint.iacr.org/2023/026>